Database Connection Pooling

PgBouncer, pool modes and sizing your pools

Watch the 4-min video

In one line: database connections are expensive, so apps reuse them in pools; every pod has its own pool, so put an external pooler like PgBouncer in front of the database, keep pools small (about 2 per database core to start) and keep the total below `max_connections`.

1 The problem: Black Friday

Your API autoscales to **200 pods**, and each pod opens a pool of **20** database connections. That is **4,000** connections, aimed at a Postgres server configured for **500**. New connections are refused, memory runs out, and the database falls over, even though every query is fast.

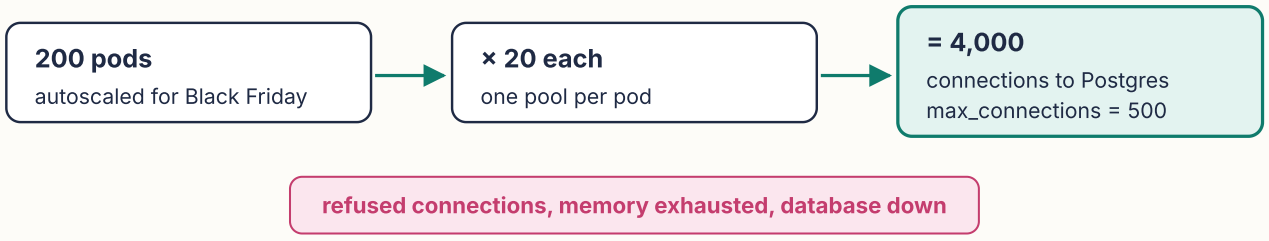


Figure 1. The pool size is per pod. Autoscaling multiplies it without anyone changing a setting.

2 Why connections are expensive

Opening a connection needs a TCP handshake, often a TLS handshake, and authentication. Together that can take **tens of milliseconds**, often longer than the query itself.

In Postgres, every connection is a **separate server process**, using several megabytes of memory. Hundreds of them means gigabytes of RAM and heavy scheduling overhead. That is why `max_connections` is usually in the hundreds (the default is 100), not the thousands.

3 Application pools, and their trap

So applications **reuse** connections. A pool library such as SQLAlchemy keeps a set of open connections. A request **borrows** one, runs its queries and **returns** it, without closing it. SQLAlchemy defaults to `pool_size=5` plus `max_overflow=10`, so up to 15 connections per process.

```
engine = create_engine(DB_URL,
    pool_size=5,        # connections kept open
    max_overflow=10,    # extra ones allowed in a burst
    pool_timeout=30,    # seconds to wait for a free one, then error
    pool_pre_ping=True) # check a connection is alive before lending it
```

The trap: **every pod (and every worker process) has its own pool**. The total grows with every pod you add, and the database has no say in it.

4 PgBouncer: an external pooler

PgBouncer sits between the apps and Postgres. It is lightweight: thousands of client connections cost it little. It **multiplexes** them onto a small number of real database connections, say 100. The database only ever sees those 100.

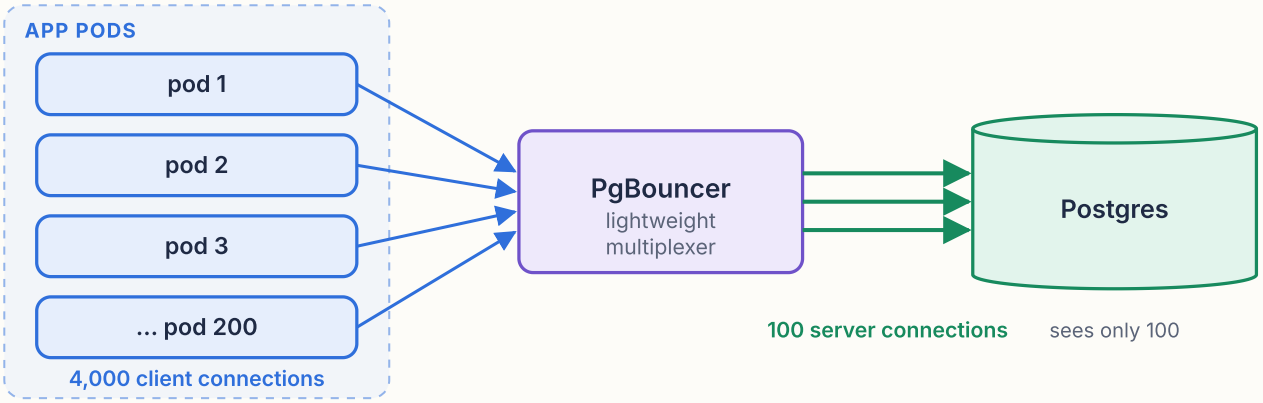


Figure 2. Apps connect to PgBouncer as if it were Postgres. PgBouncer keeps a fixed, small set of server connections.

PgBouncer has three pooling modes:

- **Session pooling:** a client keeps the same server connection for its whole session. Safe, every feature works, but it saves little, because idle clients still hold a connection.
- **Transaction pooling:** a server connection is lent only for one transaction, then returned. The most efficient mode. But **session state does not carry over**: `SET` commands, advisory locks, `LISTEN`, temporary tables (and, in older versions, prepared statements) can land on a different connection next time.
- **Statement pooling:** returned after every statement, so multi-statement transactions are not allowed. Rarely used.

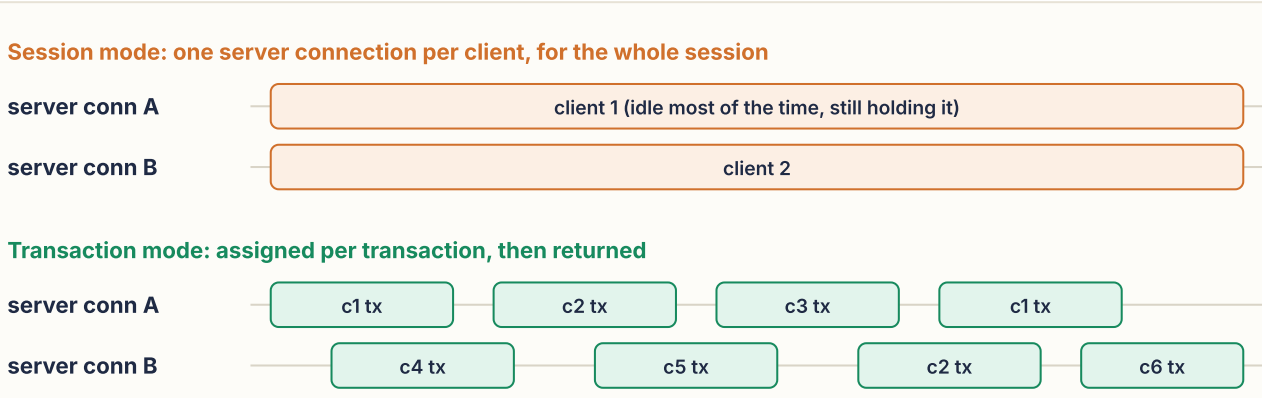


Figure 3. In transaction mode, two server connections serve six clients, because each one is busy only while a transaction runs.

5 How big should pools be? Smaller than you think

A database server can run only a few queries at once per CPU core. Beyond that, extra connections do not add work; they add **waiting and contention**. A common starting point is about **2 connections per database core**. Then measure, and adjust.

Then check the sum: **all pools across all instances (and poolers) must stay below `max_connections`**, with room left for admins, migrations, monitoring and replication.

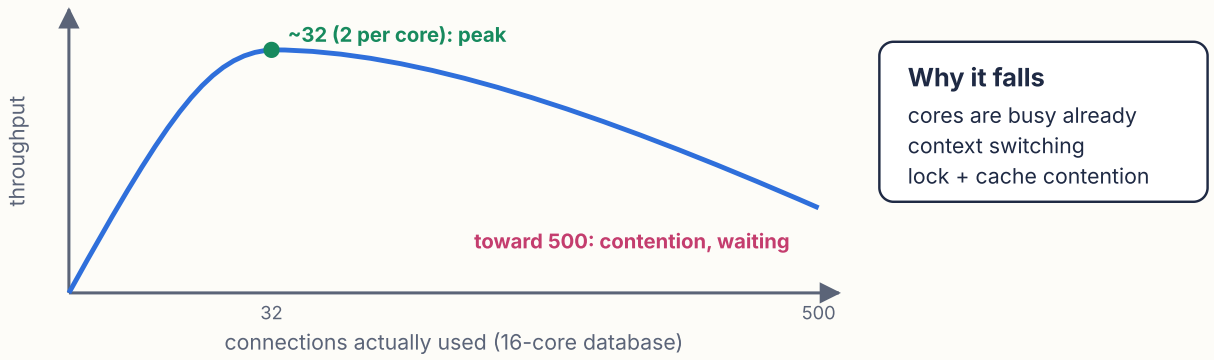


Figure 4. Throughput peaks at a small number of active connections, then falls as they fight over the same cores.

WORKED EXAMPLE: BLACK FRIDAY, FIXED

- Database: 16 cores, so start around **32** busy connections; `max_connections = 500`.
- Check with Little's Law: 4,000 queries/s × 5 ms = **20** connections busy on average, so 32 has headroom for bursts.
- 200 pods connect to **PgBouncer in transaction mode**: 4,000 client connections. PgBouncer keeps **100** server connections (room for a slower day).
- Postgres sees 100 + ~20 for admins and replicas, well under 500. The database stays healthy.

Serverless makes this worse. Each function instance may open its own connections, and a traffic spike starts thousands of instances at once. Use a pooler such as PgBouncer, or a managed proxy such as **Amazon RDS Proxy**, and keep each instance's pool at 1–2.

COMMON MISTAKES

- Sizing each pod's pool without multiplying by the maximum pod count.
- “More connections = more throughput”: past ~2 per core it gets slower.
- Using `SET`, advisory locks or `LISTEN` through transaction-mode PgBouncer.
- Leaving no spare connections for admins, migrations and replication.
- Forgetting to return connections (unclosed sessions), so the pool runs dry.

6 Key terms

Connection pool: reused open connections, borrowed per request

PgBouncer: lightweight external Postgres pooler

Transaction mode: server connection lent per transaction

max_connections: Postgres limit on connections

Multiplexing: many clients over few server connections

Statement mode: lent per statement; no transactions

Overflow: extra temporary connections in a burst

Session mode: server connection held per client session

RDS Proxy: AWS managed connection pooler

7 Interview questions

Q1. How do you size a database connection pool? BASIC

Start small, around 2 connections per database CPU core (32 for a 16-core server), measure throughput and wait time, and make sure the total across all instances stays below max_connections with room for admins and replication.

Q2. Why are database connections expensive? BASIC

TCP, often TLS, and authentication can take tens of ms, and in Postgres each connection is its own process using several MB. A few hundred of them is gigabytes of memory.

Q3. Why use PgBouncer? BASIC

It multiplexes thousands of client connections onto a few server connections, especially in transaction mode, protecting the database from connection storms caused by autoscaling and serverless.

Q4. You have 50 pods, each with SQLAlchemy defaults, and max_connections = 500. Safe? SCENARIO

Each pod can open 5 + 10 = 15, so 50 × 15 = 750 at peak, over 500. With 4 worker processes per pod it is 3,000. Shrink the pools or, better, put PgBouncer in between.

Q5. A Lambda-based API exhausts Postgres connections during a spike. Fix? SCENARIO

Each function instance opens its own connections, and a spike creates thousands of instances. Put RDS Proxy or PgBouncer in front, set the per-instance pool to 1–2, and cap function concurrency if needed.

Q6. After switching PgBouncer to transaction mode, a feature using advisory locks breaks. Why? SCENARIO

In transaction mode the next transaction can run on a different server connection, so session-level state (advisory locks, SET, LISTEN) is lost. Use transaction-scoped locks, or route that feature through a session-mode pool.

Q7. Doubling a pool from 50 to 100 made p99 latency worse. Why? TRICKY

The database was already saturated on CPU or I/O. More concurrent queries only add context switching and lock contention, so each takes longer. Smaller pools with a short queue in the app often give better throughput and latency.

Q8. Does a read replica solve a connection storm? TRICKY

Only partly. It spreads reads, but each replica has its own max_connections, and writes still hit the primary. The real fix is bounding total connections with a pooler.

Next: SD-E-030 · OLTP vs OLAP backendroadmap.com